

Theoretische Informatik

Laurin Seeholzer

September 29, 2026

Contents

1	Introduction	3
1.1	Turing Machines	3
1.2	P vs. NP	4
1.3	Polynomial Time Reductions	4
2	Finite Automata	5
2.1	Symbol, Alphabets and Words	5
2.2	Formal Languages	6
2.3	Algorithmic Problems	6
2.4	Finite Automata	6
2.5	Non-regularity	8

1 Introduction

Leibnitz proposed that some time in the future, mathematics will be developed enough to model and solve every possible problem. Then came Russel

Definition 1.1. Russels Paradox: Let S be the set of sets that do not contain themselves.

$$S = \{X \mid X \notin X\}$$

Russels Paradox poses the following Question: Does S contain itself. The answer is paradoxical

$$S \notin S \implies S \in S$$

$$S \in S \implies S \notin S$$

Remark 1.2. Hilbert style deduction allows you to solve Russels paradox by first stating what the axioms and inference rules of the System are.

Theorem 1.3. Goedels incompleteness: There are mathematical statements that are true but can not possibly be proven to be so.

1.1 Turing Machines

Definition 1.4. Turing-Machine (TM): A theoretical machine consisting of an infinity strand of paper, a perfect pen and a set of rules. Every step includes reading current position, modifying it, moving left/right.

Remark 1.5. It can be shown that every algorithm can be modeled as a Turing-Machine and vice-versa.

Theorem 1.6. Turing-Theorem: There are problems that can not be automatically solved using an algorithm/TM.

Proof. Let $w_{i,1}, w_{i,2}, w_{i,3} \dots$ be the output of Turing Machine i for inputs $1, 2, 3 \dots$. We define a new Turing Machine M_x such that $w_{x,j} \neq w_{j,j}$ hence the output of M_x differs from the output of every other Turing machine for at least one input. Observe that this is a valid Turing Machine but it can not possibly be contained in our current list of Turing Machines. Hence, we reach a contradiction: if M_x were in our enumeration at some index x , its output on input x would be $w_{x,x} \neq w_{x,x}$, which is logically impossible. Therefore, our assumption that we can construct such a machine M_x must be false. This implies there is no computable general algorithm capable of determining the output $w_{j,j}$ for all

Turing machines j , proving that there exist uncomputable functions and that the Halting Problem is undecidable. $\square$

Example 1.7. Halting-problem: Does algorithm A terminate?

This problem seems easy at first but we can design programs that for example only terminate once they find an even number n such that there are no primes p, q with $n = p + q$. Solving this (finding such n) would solve the Goldbach-Conjecture which we currently do not know to be true/false.

1.2 P vs. NP

Definition 1.8. P vs. NP: The p/np problem poses the question if all in polynomial time verifiable problems (NP) have a polynomial time solution (P).

Definition 1.9. A problem A in NP is called **NP-Complete** if the existence of an efficient solution for A implies the existence of an efficient solution of all NP problems.

1.3 Polynomial Time Reductions

Definition 1.10. A clique in a graph is a set of vertices such that they are all connected to each other.

Definition 1.11. An independent-Set in a Graph is a set of vertices, such that none of them are connected to each other.

Example 1.12. Assume we know "does G have a clique of size k " is NP-Complete. How can we show that then "Does G have a stable set of size k " is also NP-Complete.

Proof. Observe that inverting a graph (Adding edges where there are none and removing edges where there are) creates a graph G' such that every clique in G is a stable Set in G' and viceversa. This lets us calculate stable sets from cliques and vice versa hence if one problem is NP-Complete both have to be. $\square$

2 Finite Automata

2.1 Symbol, Alphabets and Words

Definition 2.1. Alphabet: finite, non-empty set.

Example 2.2. Examples of Alphabets:

- Boolean alphabet $0, 1$
- latin alphabet $a, b, \dots, z$
- logical alphabet $0, a, x, (,), \dots$

Definition 2.3. Word: A word over an alphabet A is a finite sequence of symbols of A .

Definition 2.4. The **length** of a word is the number of symbols it uses. There is always an empty word of length zero.

Remark 2.5. Words can be concatenated or reversed to form new words.

$$\begin{aligned} \text{kon}(x, y) &= xy \\ w = a_1 a_2 a_3 &\implies w^R = a_3 a_2 a_1 \end{aligned}$$

Definition 2.6. The i -th iteration of a word $x = ab$ is $x^i = ababab \dots ab$ the word concatenated with itself i times.

Definition 2.7. A word v is a partial-, prefix- or suffix-word of a word w if v is contained in w , w begins with v or ends with v .

Remark 2.8. For a word x and symbol a the expression $|x|_a$ describes the number of appearances of a in x .

Theorem 2.9. For an alphabet with an order imposed on its symbols ($a_1 \leq a_2 \leq \dots$) we can define the harmonic order of its words as a lexicographic order where

$$u \leq v \iff (|u| \leq |v|) \vee (|u| = |v| \wedge u = xs_i u', v = xs_j v'. s_i \leq s_j)$$

2.2 Formal Languages

Definition 2.10. A subset $L \subset \Sigma^*$ is called Language over the alphabet Σ

Definition 2.11. The **complement** L^C of a Language L is defined as $\Sigma^* \setminus L$

Definition 2.12. The **concatenation** of two languages L_1, L_2 is all the set of all w_1w_2 where $w_1 \in L_1, w_2 \in L_2$

Remark 2.13. We can concatenate a Language with itself (possibly multiple times or zero)

$$L^0 = \{\lambda\}, L^1 = L, L^2 = LL \dots L^* = \bigcup_{i=0}^{\infty} L^i, L^+ = LL^*$$

Lemma 2.14. Let L_1, L_2, L_3 be languages over an alphabet Σ then

$$L_1L_2 \cup L_1L_3 = L_1(L_2 \cup L_3)$$

$$L_1(L_2 \cap L_3) \subseteq L_1L_2 \cap L_1L_3$$

2.3 Algorithmic Problems

Remark 2.15. An algorithm describes a function $\Sigma_{in}^* \rightarrow \Sigma_{out}^*$

Definition 2.16. Let Σ be an alphabet and $L \subseteq \Sigma^*$. The decision problem (Σ, L) is defined (for arb $x \in \Sigma^*$) as

$$x \in L \quad \text{or} \quad x \notin L$$

Remark 2.17. If an algorithm A correctly decides for any x then we say

$$A \quad \text{detects the Language} \quad L$$

2.4 Finite Automata

Remark 2.18. A finite automata (finite state machine) is a set of states together with state transitions on inputs with fixed boolean outputs on each state.

Definition 2.19. Finite Automata: is a Quadroupel $M = (Q, \Sigma, \delta, q_0, F)$

- Q is a finite set of states
- Σ is an alphabet (Input-alphabet)
- δ is a function $Q \times \Sigma \rightarrow Q$
- $q_0 \in Q$ is the starting-state
- $F \subseteq Q$ is the set of accepting (true) states

Definition 2.20. Configuration: of an Automata is an Element of $Q \times \Sigma^*$

- **Starting-Configuraiton:** $C_0 = (q_0, x) | x \in \Sigma^*$
- **End-Configruation:** $C_n = (q_0, \lambda)$ where its accepting if $C_n \in F \times \{\lambda\}$

Remark 2.21. A calculation can be seen as a finite sequence of configurations $C_1, C_2 \dots$ such that for every $C_i \rightarrow_{s_M} C_{i+1}$ with steps s_M in our machine M

$$(q_0, 01001) \vdash_M (q_x, 1001) \vdash_M (q_y, 001) \dots$$

where a step is a relation on configurations

$$\vdash_M \subseteq (Q \times \Sigma^*) \times (Q \times \Sigma^*)$$

Definition 2.22. The **transitive hull of transitions** describes the calculation over multiple steps:

$$(q, w) \vdash_M^* (p, u) \iff (q = p \wedge w = u) \vee \exists k \in \mathbb{N}^+ \text{ s.t.}$$

$$w = a_1 \dots a_k u \quad \text{with} \quad a_i \in \Sigma \quad \text{and}$$

$$\exists r_1, \dots, r_{k-1} \in Q \text{ s.t. } (q, w) \vdash_M (r_1, a_2 \dots a_k u) \vdash_M (r_2, a_3 \dots a_k u) \dots \vdash_M (p, u)$$

Definition 2.23. The **extended transition function** evaluates the end state of a word over multiple transitions:

$$\tilde{\delta} : Q \times \Sigma^* \rightarrow Q$$

$$\tilde{\delta}(q, \lambda) = q \quad \forall q \in Q$$

$$\tilde{\delta}(q, wa) = \delta(\tilde{\delta}(q, w), a) \quad \forall a \in \Sigma, w \in \Sigma^*, q \in Q$$

Remark 2.24. States can be grouped into equivalence classes containing all elements with the same end state. This is useful for proofs regarding the language of finite automata, which are typically done through these classes and induction.

Definition 2.25. Product Automata: Used to determine the intersection, union or exclusion ($\odot$) of two finite automata (their languages), built with the state space $Q = Q_1 \times Q_2$.

$$\begin{aligned} M_1 &= (Q_1, \Sigma, \delta_1, q_{01}, F_1), \quad M_2 = (Q_2, \Sigma, \delta_2, q_{02}, F_2) \\ \implies M &= (Q_1 \times Q_2, \delta((x, y), a) = (\delta_1(x, a), \delta_2(y, a)), (q_{01}, q_{02}), F_1 \odot F_2) \end{aligned}$$

Remark 2.26. The correctness of this product automata can be proven by induction over the word length.

2.5 Non-regularity

Definition 2.27. A language L is called **regular** if there exists a finite automata that accepts/detects the language L and **non-regular** else.

Lemma 2.28. Let $A = (Q, \Sigma, \delta, q_0, F)$ be a finite automata and let $x, y \in \Sigma^*$ such that

$$\tilde{\delta}(q_0, x) = \tilde{\delta}(q_0, y)$$

then there exists $z \in \Sigma^*$ such that

$$Kl[xz] = Kl[yz]$$

Remark 2.29. This illustrates that finite machines are finite hence can not distinguish between xz and yz once they are in a certain state q_r , they do not know how they got there.

Proof. Proof of Non-Regularity: Let $L = \{0^n 1^n | n \in \mathbb{N}\}$ and assume $L \in L_{EA}$ (L is regular) hence there exists a finite automata A such that $L(A) = L$. Let $m = |Q|$ and observe the words $x \in \{0^n | n \leq m + 1\}$. Because this are more words than A has states there exists $i \neq j \in [0, m + 1] \cap \mathbb{N}$ such that

$$\tilde{\delta}(q_0, 0^i) = \tilde{\delta}(q_0, 0^j)$$

Hence by the above lemma the words $0^i 1^i, 0^j 1^i$ will both be accepted which contradicts the assumption that A detects L because $0^j 1^i \notin L$. $\square$